

In [4]: `# churn_prediction_app/app.py`

```
import streamlit as st
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
import joblib
```

In [3]: `#pip install streamlit`

```
0:07 ----- 2.2/9.9 MB 1.2 MB/s eta 0:0
0:07 ----- 2.3/9.9 MB 1.2 MB/s eta 0:0
0:07 ----- 2.4/9.9 MB 1.2 MB/s eta 0:0
0:07 ----- 2.4/9.9 MB 1.3 MB/s eta 0:0
0:06 ----- 2.5/9.9 MB 1.3 MB/s eta 0:0
0:06 ----- 2.7/9.9 MB 1.3 MB/s eta 0:0
0:06 ----- 2.7/9.9 MB 1.3 MB/s eta 0:0
0:06 ----- 2.9/9.9 MB 1.4 MB/s eta 0:0
0:06 ----- 3.1/9.9 MB 1.4 MB/s eta 0:0
0:05 ----- 3.2/9.9 MB 1.5 MB/s eta 0:0
```

In [6]: `data=pd.read_csv('customer_churn_dataset-testing-master.csv')`
data

Out[6]:

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type	Conti Len
0	1	22	Female	25	14	4	27	Basic	Mon
1	2	41	Female	28	28	7	13	Standard	Mon
2	3	47	Male	27	10	2	29	Premium	Anr
3	4	35	Male	9	12	5	17	Premium	Quart
4	5	53	Female	58	24	9	2	Standard	Anr
...	...	...	...	...	...	...	...	...	...
64369	64370	45	Female	33	12	6	21	Basic	Quart
64370	64371	37	Male	6	1	5	22	Standard	Anr
64371	64372	25	Male	39	14	8	30	Premium	Mon
64372	64373	50	Female	18	19	7	22	Standard	Mon
64373	64374	52	Female	45	15	9	25	Standard	Mon

64374 rows × 12 columns

```
In [7]: data.isnull().sum()
```

```
Out[7]: CustomerID      0  
        Age            0  
        Gender         0  
        Tenure         0  
        Usage Frequency 0  
        Support Calls   0  
        Payment Delay   0  
        Subscription Type 0  
        Contract Length 0  
        Total Spend     0  
        Last Interaction 0  
        Churn           0  
        dtype: int64
```

```
In [ ]: # churn_prediction_app/churn_predict_jupyter.py (for Jupyter Notebook usage

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
import joblib
import shap
import matplotlib.pyplot as plt

# Upload CSV manually or via notebook cell
file_path = 'customer_churn_dataset-testing-master.csv' # Replace with your file path

# Load dataset
df = pd.read_csv(file_path)
print("Data Preview:")
print(df.head())

# Drop CustomerID if present
if 'CustomerID' in df.columns:
    df.drop(columns=['CustomerID'], inplace=True)

# Check for class imbalance
print("\nClass Distribution:")
print(df['Churn'].value_counts(normalize=True))

# Check for potential data leakage
print("\nCorrelation with target:")
print(df.corr(numeric_only=True)['Churn'].sort_values(ascending=False))

# Remove highly correlated features to prevent leakage
leakage_features = ['Payment Delay', 'Support Calls'] # example high correlation features
for feature in leakage_features:
    if feature in df.columns:
        df.drop(columns=[feature], inplace=True)

# Define target and features
target_col = 'Churn'
features = [col for col in df.columns if col != target_col]

X = df[features]
y = df[target_col]

# One-hot encode categorical variables
X = pd.get_dummies(X)

# Align features if needed
X, y = X.align(y, axis=0)

# Train-test split with stratification
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# Train model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Evaluate on test set
y_pred = model.predict(X_test)
print("\nModel Evaluation:")
```

```
print(classification_report(y_test, y_pred))

# Cross-validation to verify generalization
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring='accuracy')
print("\nCross-Validation Accuracy (5-fold):", scores)
print("Average CV Accuracy:", np.mean(scores))

# Save model and feature list
joblib.dump(model, "model.pkl")
joblib.dump(X.columns.tolist(), "features.pkl")
print("\n✅ Model trained and saved!")

# Feature importance using SHAP
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_train)

print("\n🔍 SHAP summary plot for feature importance:")
shap.summary_plot(shap_values[1], X_train)

# --- Predict on new data (optional) ---
# new_data_path = 'new_customers.csv' # Replace with your new data CSV
# new_df = pd.read_csv(new_data_path)
# if 'CustomerID' in new_df.columns:
#     new_df.drop(columns=['CustomerID'], inplace=True)
# new_df = new_df.drop(columns=leakage_features, errors='ignore')
# new_X = pd.get_dummies(new_df)
# new_X = new_X.reindex(columns=X.columns, fill_value=0)
# predictions = model.predict_proba(new_X)[: , 1]
# new_df['Churn Probability'] = predictions
# new_df.to_csv("predictions.csv", index=False)
# print("\n📄 Predictions saved to predictions.csv")
```

Data Preview:

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	\
0	1	22	Female	25	14	4	
1	2	41	Female	28	28	7	
2	3	47	Male	27	10	2	
3	4	35	Male	9	12	5	
4	5	53	Female	58	24	9	

	Payment Delay	Subscription Type	Contract Length	Total Spend	\
0	27	Basic	Monthly	598	
1	13	Standard	Monthly	584	
2	29	Premium	Annual	757	
3	17	Premium	Quarterly	232	
4	2	Standard	Annual	533	

	Last Interaction	Churn
0	9	1
1	20	0
2	21	0
3	18	0
4	18	0

Class Distribution:

Churn

0 0.526315

1 0.473685

Name: proportion, dtype: float64

Correlation with target:

Churn	1.000000
Payment Delay	0.557386
Support Calls	0.304631
Tenure	0.195327
Age	0.063457
Last Interaction	-0.002818
Total Spend	-0.078867
Usage Frequency	-0.115098

Name: Churn, dtype: float64

Model Evaluation:

	precision	recall	f1-score	support
0	0.68	0.63	0.66	6776
1	0.62	0.67	0.65	6099
accuracy			0.65	12875
macro avg	0.65	0.65	0.65	12875
weighted avg	0.65	0.65	0.65	12875

Cross-Validation Accuracy (5-fold): [0.63961165 0.64442718 0.64170874 0.64427184 0.65263321]

Average CV Accuracy: 0.6445305263411919

✅ Model trained and saved!

In []: `#pip install joblib`